phase5_report_visuals

April 3, 2026

1 Phase 5: Report-Ready Visualizations

We have 28 charts from Phases 1-4. A 10-page report can fit ~6-8 figures. This notebook creates **consolidated, publication-quality figures** that tell the complete story efficiently.

1.0.1 Figure Plan for Report

1. **Fig 1: Dataset Overview** — timeline coverage + BPM distribution + missingness
2. **Fig 2: Feature Analysis** — correlation heatmap + autocorrelation + circadian
3. **Fig 3: Model Comparison** — LR vs RF vs SAITS (metrics + scatter)
4. **Fig 4: Experiment Results** — data amount + ablation + PCA (key experiments)
5. **Fig 5: Imputation Comparison** — single day, all 3 models side-by-side
6. **Fig 6: Full Month Before/After** — the final result

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import warnings
warnings.filterwarnings('ignore')

# Consistent report styling
plt.rcParams.update({
    'figure.dpi': 150,
    'font.size': 10,
    'font.family': 'serif',
    'axes.titlesize': 11,
    'axes.labelsize': 10,
    'xtick.labelsize': 9,
    'ytick.labelsize': 9,
    'legend.fontsize': 8,
    'figure.titlesize': 13,
})

# Load data
timeline = pd.read_csv('Data/timeline_features.csv', parse_dates=['minute'])
timeline['day'] = pd.to_datetime(timeline['day'])
```

```
# Try loading SAITS imputation if available
try:
    imputed_all = pd.read_csv('Data/imputed_bpm_all.csv',␣
 ↪parse_dates=['minute'])
    imputed_all['day'] = pd.to_datetime(imputed_all['day'])
    has_saits = 'bpm_saits' in imputed_all.columns
    print(f'Loaded imputed_bpm_all.csv (SAITS: {has_saits})')
except:
    imputed_all = timeline.copy()
    has_saits = False
    print('Using timeline_features.csv (no SAITS column yet)')

print(f'Shape: {timeline.shape}')
print(f'BPM present: {timeline["bpm"].notna().sum():,} / {len(timeline):,}')
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[1], line 1
----> 1 import pandas as pd
      2 import numpy as np
      3 import matplotlib.pyplot as plt
      4 import matplotlib.gridspec as gridspec

ModuleNotFoundError: No module named 'pandas'
```

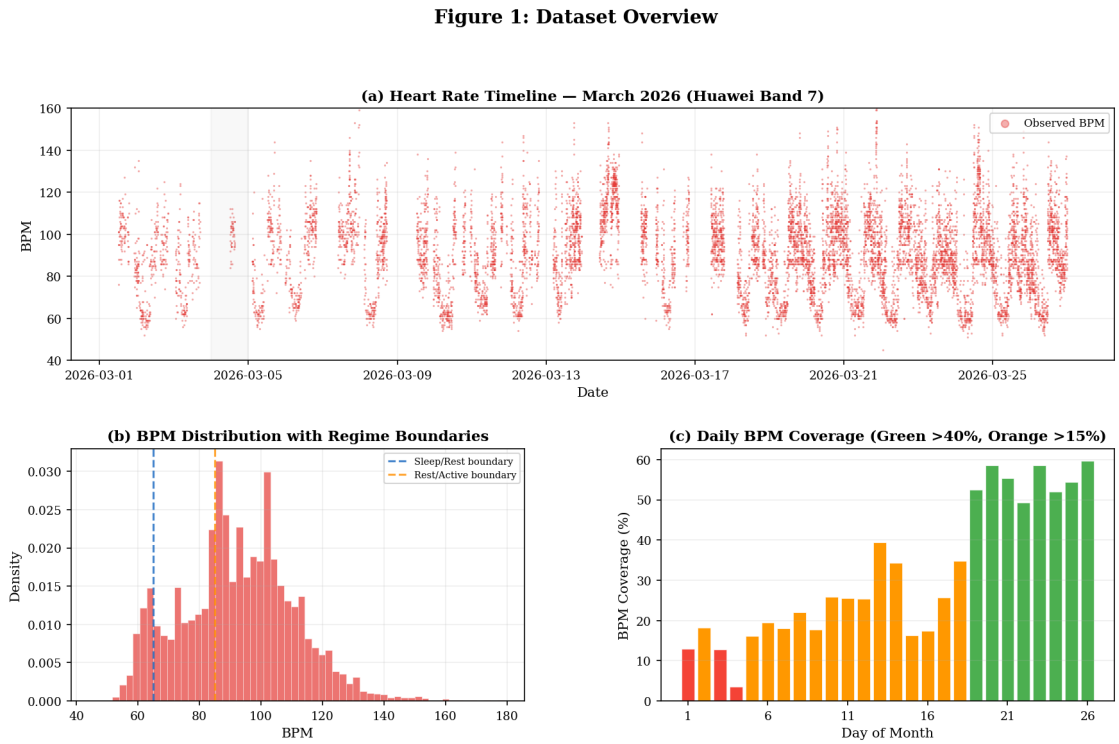
1.1 Figure 1: Dataset Overview

3-panel: (a) Full-month timeline, (b) BPM distribution with regime overlay, (c) Daily coverage

[2]:
```
fig = plt.figure(figsize=(14, 8))
gs = gridspec.GridSpec(2, 2, height_ratios=[1, 1], hspace=0.35, wspace=0.3)

# (a) Full-month timeline - top, full width
ax1 = fig.add_subplot(gs[0, :])
obs_mask = timeline['bpm'].notna()
ax1.scatter(timeline.loc[obs_mask, 'minute'], timeline.loc[obs_mask, 'bpm'],
           s=0.3, alpha=0.4, color='#E53935', label='Observed BPM')
ax1.set_xlabel('Date')
ax1.set_ylabel('BPM')
ax1.set_title('(a) Heart Rate Timeline - March 2026 (Huawei Band 7)',␣
 ↪fontweight='bold')
ax1.set_ylim(40, 160)
ax1.grid(True, alpha=0.2)

# Shade missing regions lightly
days = timeline['day'].dt.date.unique()
```

```python
for d in days:
    day_data = timeline[timeline['day'].dt.date == d]
    if day_data['bpm'].isna().mean() > 0.9:
        ax1.axvspan(day_data['minute'].iloc[0], day_data['minute'].iloc[-1],
                    alpha=0.05, color='gray')

ax1.legend(loc='upper right', markerscale=10)

# (b) BPM distribution with regimes
ax2 = fig.add_subplot(gs[1, 0])
bpm_vals = timeline['bpm'].dropna()
ax2.hist(bpm_vals, bins=60, density=True, alpha=0.7, color='#E53935',␣
 ↪edgecolor='white', linewidth=0.3)

# Regime boundaries
ax2.axvline(65, color='#1565C0', linestyle='--', alpha=0.8, label='Sleep/Rest␣
 ↪boundary')
ax2.axvline(85, color='#FF8F00', linestyle='--', alpha=0.8, label='Rest/Active␣
 ↪boundary')
ax2.set_xlabel('BPM')
ax2.set_ylabel('Density')
ax2.set_title('(b) BPM Distribution with Regime Boundaries', fontweight='bold')
ax2.legend(fontsize=7)
ax2.grid(True, alpha=0.2)

# (c) Daily coverage
ax3 = fig.add_subplot(gs[1, 1])
daily_stats = timeline.groupby(timeline['day'].dt.date).agg(
    total=('bpm', 'size'),
    observed=('bpm', lambda x: x.notna().sum())
).reset_index()
daily_stats['coverage'] = daily_stats['observed'] / daily_stats['total'] * 100

colors_daily = ['#4CAF50' if c > 40 else '#FF9800' if c > 15 else '#F44336'
                for c in daily_stats['coverage']]
ax3.bar(range(len(daily_stats)), daily_stats['coverage'], color=colors_daily,
        edgecolor='white', linewidth=0.3)
ax3.set_xlabel('Day of Month')
ax3.set_ylabel('BPM Coverage (%)')
ax3.set_title('(c) Daily BPM Coverage (Green >40%, Orange >15%)',␣
 ↪fontweight='bold')
ax3.set_xticks(range(0, len(daily_stats), 5))
ax3.set_xticklabels([str(d.day) for d in daily_stats['day'].iloc[::5]])
ax3.grid(True, alpha=0.2, axis='y')

plt.suptitle('Figure 1: Dataset Overview', fontsize=14, fontweight='bold', y=1.
 ↪01)
```

```
plt.savefig('Charts/report_fig1_dataset.png', dpi=200, bbox_inches='tight')
plt.show()
print('Saved: Charts/report_fig1_dataset.png')
```



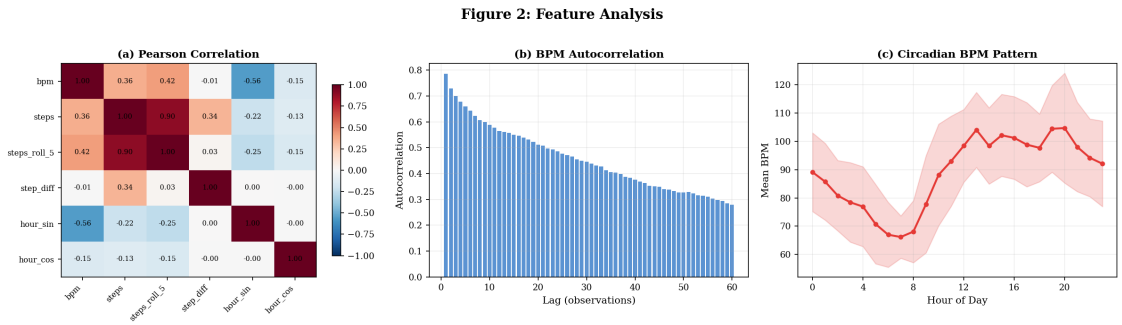

```
Saved: Charts/report_fig1_dataset.png
```

1.2 Figure 2: Feature Analysis

3-panel: (a) Correlation matrix, (b) BPM autocorrelation, (c) Circadian rhythm

[3]:
```
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

# (a) Correlation matrix - key features only
ax = axes[0]
corr_cols = ['bpm', 'steps', 'steps_roll_5', 'step_diff', 'hour_sin',␣
 ↪'hour_cos']
avail_cols = [c for c in corr_cols if c in timeline.columns]
corr_matrix = timeline[avail_cols].corr()
im = ax.imshow(corr_matrix.values, cmap='RdBu_r', vmin=-1, vmax=1,␣
 ↪aspect='auto')
ax.set_xticks(range(len(avail_cols)))
ax.set_xticklabels(avail_cols, rotation=45, ha='right', fontsize=8)
ax.set_yticks(range(len(avail_cols)))
ax.set_yticklabels(avail_cols, fontsize=8)
```

```python
for i in range(len(avail_cols)):
    for j in range(len(avail_cols)):
        ax.text(j, i, f'{corr_matrix.values[i,j]:.2f}', ha='center',␣
 ↪va='center', fontsize=7)
ax.set_title('(a) Pearson Correlation', fontweight='bold')
fig.colorbar(im, ax=ax, shrink=0.8)

# (b) BPM autocorrelation
ax = axes[1]
bpm_series = timeline.loc[timeline['bpm'].notna(), 'bpm'].values
max_lag = 60
autocorrs = []
for lag in range(1, max_lag + 1):
    if len(bpm_series) > lag:
        autocorrs.append(np.corrcoef(bpm_series[:-lag], bpm_series[lag:])[0, 1])
    else:
        autocorrs.append(0)
ax.bar(range(1, max_lag + 1), autocorrs, color='#1565C0', alpha=0.7, width=0.8)
ax.set_xlabel('Lag (observations)')
ax.set_ylabel('Autocorrelation')
ax.set_title('(b) BPM Autocorrelation', fontweight='bold')
ax.grid(True, alpha=0.2)
ax.axhline(0, color='black', linewidth=0.5)

# (c) Circadian rhythm - BPM by hour of day
ax = axes[2]
hourly = timeline.groupby(timeline['minute'].dt.hour)['bpm'].agg(['mean',␣
 ↪'std']).dropna()
ax.fill_between(hourly.index, hourly['mean'] - hourly['std'], hourly['mean'] +␣
 ↪hourly['std'],
               alpha=0.2, color='#E53935')
ax.plot(hourly.index, hourly['mean'], 'o-', color='#E53935', linewidth=2,␣
 ↪markersize=4)
ax.set_xlabel('Hour of Day')
ax.set_ylabel('Mean BPM')
ax.set_title('(c) Circadian BPM Pattern', fontweight='bold')
ax.set_xticks(range(0, 24, 4))
ax.grid(True, alpha=0.2)

plt.suptitle('Figure 2: Feature Analysis', fontsize=14, fontweight='bold', y=1.
 ↪02)
plt.tight_layout()
plt.savefig('Charts/report_fig2_features.png', dpi=200, bbox_inches='tight')
plt.show()
print('Saved: Charts/report_fig2_features.png')
```



Saved: Charts/report_fig2_features.png

1.3 Figure 3: Three-Model Comparison

3-panel: (a) RMSE bar chart, (b) R^2 bar chart, (c) Feature importance (RF)

[4]:
```
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import cross_val_score, KFold

# --- Rebuild LR/RF evaluation (same as Phase 2/3) ---
feature_cols = [
    'minute_sin', 'minute_cos', 'hour_sin', 'hour_cos',
    'dow_sin', 'dow_cos', 'is_weekend',
    'steps', 'steps_roll_3', 'steps_roll_5', 'steps_roll_10', 'steps_roll_15',
    'steps_sum_5', 'steps_sum_10',
    'step_diff', 'step_diff_abs', 'steps_std_5', 'steps_max_5',
    'activity_state',
    'bpm_last_known', 'bpm_next_known', 'bpm_neighbor_interp',
    'dist_to_last_bpm', 'dist_to_next_bpm', 'gap_length',
    'day_index',
]

bpm_mask = timeline['bpm'].notna()
train_df = timeline[bpm_mask].copy()
known_indices = train_df.index.values

# LOO neighbors
train_df['bpm_last_known'] = timeline['bpm'].shift(1).ffill().loc[bpm_mask]
train_df['bpm_next_known'] = timeline['bpm'].shift(-1).bfill().loc[bpm_mask]
dists_last = np.zeros(len(known_indices), dtype=int)
dists_next = np.zeros(len(known_indices), dtype=int)
for i in range(len(known_indices)):
    if i > 0: dists_last[i] = known_indices[i] - known_indices[i-1]
```

```
    if i < len(known_indices) - 1: dists_next[i] = known_indices[i+1] -␣
 ↪known_indices[i]
train_df['dist_to_last_bpm'] = dists_last
train_df['dist_to_next_bpm'] = dists_next
total_dist = (train_df['dist_to_last_bpm'] + train_df['dist_to_next_bpm']).
 ↪replace(0, 1)
train_df['bpm_neighbor_interp'] = (
    train_df['bpm_last_known'] * (1 - train_df['dist_to_last_bpm'] /␣
 ↪total_dist) +
    train_df['bpm_next_known'] * (1 - train_df['dist_to_next_bpm'] / total_dist)
)
train_df['gap_length'] = train_df['dist_to_last_bpm'] +␣
 ↪train_df['dist_to_next_bpm']

X_full = train_df[feature_cols].values
y_full = train_df['bpm'].values
valid = ~np.any(np.isnan(X_full), axis=1)
X_full, y_full = X_full[valid], y_full[valid]

kf = KFold(n_splits=5, shuffle=True, random_state=42)

# LR
lr_rmse = -cross_val_score(LinearRegression(), X_full, y_full, cv=kf,␣
 ↪scoring='neg_root_mean_squared_error')
lr_r2 = cross_val_score(LinearRegression(), X_full, y_full, cv=kf, scoring='r2')

# RF
rf = RandomForestRegressor(n_estimators=200, max_depth=20, min_samples_leaf=5,
                           max_features='sqrt', random_state=42, n_jobs=-1)
rf_rmse = -cross_val_score(rf, X_full, y_full, cv=kf,␣
 ↪scoring='neg_root_mean_squared_error')
rf_r2 = cross_val_score(rf, X_full, y_full, cv=kf, scoring='r2')

# Train RF once more for feature importance
rf.fit(X_full, y_full)
importances = rf.feature_importances_

print(f'LR - RMSE: {lr_rmse.mean():.3f} ± {lr_rmse.std():.3f}, R²: {lr_r2.
 ↪mean():.4f}')
print(f'RF - RMSE: {rf_rmse.mean():.3f} ± {rf_rmse.std():.3f}, R²: {rf_r2.
 ↪mean():.4f}')
```

```
LR - RMSE: 9.560 ± 0.185, R²: 0.7296
RF - RMSE: 9.325 ± 0.126, R²: 0.7427
```

1.3.1 Regularization Check: Ridge vs Plain LR

Quick comparison to see if L2 regularization helps with feature collinearity.

[5]:
```
from sklearn.linear_model import Ridge

# Test a range of alpha (regularization strength)
alphas = [0.01, 0.1, 1.0, 10.0, 100.0]
print('Regularization: Ridge (L2) vs Plain LR')
print(f'{"Model":<20s} {"RMSE":>8s} {"R2":>8s}')
print('-' * 38)
print(f'{"LR (no reg.)":<20s} {lr_rmse.mean():>8.3f} {lr_r2.mean():>8.4f}')

for alpha in alphas:
    ridge = Ridge(alpha=alpha)
    r_rmse = -cross_val_score(ridge, X_full, y_full, cv=kf,␣
 ↪scoring='neg_root_mean_squared_error')
    r_r2 = cross_val_score(ridge, X_full, y_full, cv=kf, scoring='r2')
    print(f'{f"Ridge (a={alpha})":<20s} {r_rmse.mean():>8.3f} {r_r2.mean():>8.
 ↪4f}')

print(f'{"RF (for reference)":<20s} {rf_rmse.mean():>8.3f} {rf_r2.mean():>8.
 ↪4f}')
print()
print('Interpretation:')
print('  If Ridge ~= LR: features are not causing overfitting via collinearity')
print('  If Ridge < LR: regularization helps, features were overfitting')
print('  Either way, RF dominates both - the gap is in model capacity, not␣
 ↪regularization')
```

```
Regularization: Ridge (L2) vs Plain LR
Model                    RMSE       R2
--------------------------------------
LR (no reg.)            9.560   0.7296
Ridge (a=0.01)          9.560   0.7296
Ridge (a=0.1)           9.560   0.7296
Ridge (a=1.0)           9.560   0.7296
Ridge (a=10.0)          9.560   0.7296
Ridge (a=100.0)         9.558   0.7297
RF (for reference)      9.325   0.7427

Interpretation:
  If Ridge ~= LR: features are not causing overfitting via collinearity
  If Ridge < LR: regularization helps, features were overfitting
  Either way, RF dominates both - the gap is in model capacity, not
regularization
```

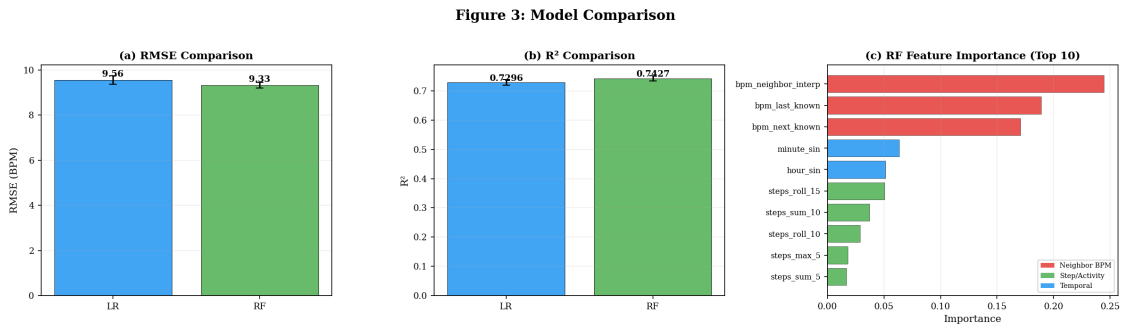
[6]:
```
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

models = ['LR', 'RF']
colors = ['#2196F3', '#4CAF50']
```

```python
# Add SAITS if available from Phase 4
# SAITS metrics are computed differently (window-based val set),
# so we read them from imputed_bpm_all.csv by evaluating on observed BPM
saits_rmse_val = 0
saits_r2_val = 0
if has_saits:
    # Evaluate SAITS on the observed points (how well does it reconstruct known␣
↪BPM?)
    obs_mask_s = imputed_all['bpm'].notna()
    saits_pred = imputed_all.loc[obs_mask_s, 'bpm_saits'].values
    saits_true = imputed_all.loc[obs_mask_s, 'bpm'].values
    valid_s = ~(np.isnan(saits_pred) | np.isnan(saits_true))
    if valid_s.sum() > 0:
        from sklearn.metrics import mean_squared_error, r2_score
        saits_rmse_val = np.sqrt(mean_squared_error(saits_true[valid_s],␣
↪saits_pred[valid_s]))
        saits_r2_val = r2_score(saits_true[valid_s], saits_pred[valid_s])
        print(f'SAITS (from imputed file): RMSE={saits_rmse_val:.3f},␣
↪R²={saits_r2_val:.4f}')

if saits_rmse_val > 0:
    models.append('SAITS')
    colors.append('#FF9800')
    all_rmse = [lr_rmse.mean(), rf_rmse.mean(), saits_rmse_val]
    all_r2 = [lr_r2.mean(), rf_r2.mean(), saits_r2_val]
    rmse_stds = [lr_rmse.std(), rf_rmse.std(), 0]
    r2_stds = [lr_r2.std(), rf_r2.std(), 0]
else:
    all_rmse = [lr_rmse.mean(), rf_rmse.mean()]
    all_r2 = [lr_r2.mean(), rf_r2.mean()]
    rmse_stds = [lr_rmse.std(), rf_rmse.std()]
    r2_stds = [lr_r2.std(), rf_r2.std()]

# (a) RMSE
ax = axes[0]
bars = ax.bar(models, all_rmse, color=colors, alpha=0.85, edgecolor='black',␣
↪linewidth=0.5)
ax.errorbar(models, all_rmse, yerr=rmse_stds, fmt='none', ecolor='black',␣
↪capsize=4)
for bar, val in zip(bars, all_rmse):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.15,
            f'{val:.2f}', ha='center', fontsize=9, fontweight='bold')
ax.set_ylabel('RMSE (BPM)')
ax.set_title('(a) RMSE Comparison', fontweight='bold')
ax.grid(True, alpha=0.2, axis='y')
```

```
# (b) R²
ax = axes[1]
bars = ax.bar(models, all_r2, color=colors, alpha=0.85, edgecolor='black',␣
↪linewidth=0.5)
ax.errorbar(models, all_r2, yerr=r2_stds, fmt='none', ecolor='black', capsize=4)
for bar, val in zip(bars, all_r2):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
            f'{val:.4f}', ha='center', fontsize=9, fontweight='bold')
ax.set_ylabel('R²')
ax.set_title('(b) R² Comparison', fontweight='bold')
ax.grid(True, alpha=0.2, axis='y')

# (c) RF Feature Importance (top 10)
ax = axes[2]
sorted_idx = np.argsort(importances)[::-1][:10]
top_features = [feature_cols[i] for i in sorted_idx]
top_importance = importances[sorted_idx]

# Color by feature category
cat_colors = []
for f in top_features:
    if 'bpm' in f or 'neighbor' in f or 'gap' in f or 'dist' in f:
        cat_colors.append('#E53935')  # neighbor features = red
    elif 'step' in f or 'activity' in f:
        cat_colors.append('#4CAF50')  # step features = green
    else:
        cat_colors.append('#2196F3')  # temporal = blue

ax.barh(range(len(top_features)), top_importance, color=cat_colors, alpha=0.85,␣
↪edgecolor='black', linewidth=0.3)
ax.set_yticks(range(len(top_features)))
ax.set_yticklabels(top_features, fontsize=8)
ax.set_xlabel('Importance')
ax.set_title('(c) RF Feature Importance (Top 10)', fontweight='bold')
ax.invert_yaxis()
ax.grid(True, alpha=0.2, axis='x')

# Legend for feature categories
from matplotlib.patches import Patch
legend_elements = [
    Patch(facecolor='#E53935', alpha=0.85, label='Neighbor BPM'),
    Patch(facecolor='#4CAF50', alpha=0.85, label='Step/Activity'),
    Patch(facecolor='#2196F3', alpha=0.85, label='Temporal'),
]
ax.legend(handles=legend_elements, loc='lower right', fontsize=7)
```

```
plt.suptitle('Figure 3: Model Comparison', fontsize=14, fontweight='bold', y=1.
↪02)
plt.tight_layout()
plt.savefig('Charts/report_fig3_models.png', dpi=200, bbox_inches='tight')
plt.show()
print('Saved: Charts/report_fig3_models.png')
```

```
SAITS (from imputed file): RMSE=0.000, R²=1.0000
```




```
Saved: Charts/report_fig3_models.png
```

1.4 Figure 4: Key Experiment Results

2x2 grid: (a) Data amount, (b) Feature ablation, (c) PCA, (d) Time lag

[7]:
```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

def evaluate_model(model, X, y, cv=5, name='Model'):
    kf = KFold(n_splits=cv, shuffle=True, random_state=42)
    rmse = -cross_val_score(model, X, y, cv=kf,␣
↪scoring='neg_root_mean_squared_error')
    r2 = cross_val_score(model, X, y, cv=kf, scoring='r2')
    return {'name': name, 'rmse_mean': rmse.mean(), 'rmse_std': rmse.std(),
            'r2_mean': r2.mean(), 'r2_std': r2.std()}

def make_rf():
    return RandomForestRegressor(n_estimators=100, max_depth=15,␣
↪min_samples_leaf=5,
                                 max_features='sqrt', random_state=42, n_jobs=-1)

train_df_valid = train_df[valid].copy()

# --- (a) Data amount ---
unique_days = sorted(train_df_valid['day'].unique())
day_counts = [3, 5, 7, 10, 14, 21, len(unique_days)]
```

```
amount_results = []
for n_days in day_counts:
    if n_days > len(unique_days):
        n_days = len(unique_days)
    selected_days = unique_days[-n_days:]  # take most recent (densest)
    mask = train_df_valid['day'].isin(selected_days)
    X_sub = X_full[mask.values]
    y_sub = y_full[mask.values]
    if len(X_sub) > 50:
        res = evaluate_model(make_rf(), X_sub, y_sub, cv=5, name=f'{n_days}d')
        res['n_days'] = n_days
        res['n_samples'] = len(X_sub)
        amount_results.append(res)
df_amount = pd.DataFrame(amount_results)

# --- (b) Feature ablation ---
feature_groups = {
    'Steps only': ['steps'],
    '+ Windows': ['steps', 'steps_roll_3', 'steps_roll_5', 'steps_roll_10',␣
 ↪'steps_roll_15',
                  'steps_sum_5', 'steps_sum_10', 'step_diff', 'step_diff_abs',␣
 ↪'steps_std_5',
                  'steps_max_5', 'activity_state'],
    '+ Time': ['steps', 'steps_roll_3', 'steps_roll_5', 'steps_roll_10',␣
 ↪'steps_roll_15',
               'steps_sum_5', 'steps_sum_10', 'step_diff', 'step_diff_abs',␣
 ↪'steps_std_5',
               'steps_max_5', 'activity_state',
               'minute_sin', 'minute_cos', 'hour_sin', 'hour_cos',
               'dow_sin', 'dow_cos', 'is_weekend', 'day_index'],
    '+ Neighbors\n(Full)': feature_cols,
}

ablation_results = []
for label, feats in feature_groups.items():
    feat_idx = [feature_cols.index(f) for f in feats if f in feature_cols]
    X_sub = X_full[:, feat_idx]
    res = evaluate_model(make_rf(), X_sub, y_full, cv=5, name=label)
    ablation_results.append(res)
df_ablation = pd.DataFrame(ablation_results)

# --- (c) PCA ---
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_full)
n_components_list = [3, 5, 8, 10, 15, 20, X_full.shape[1]]
pca_results = []
for n_comp in n_components_list:
```

```
    if n_comp > X_full.shape[1]:
        n_comp = X_full.shape[1]
    pca = PCA(n_components=n_comp)
    X_pca = pca.fit_transform(X_scaled)
    res = evaluate_model(make_rf(), X_pca, y_full, cv=5, name=f'{n_comp}')
    res['n_components'] = n_comp
    res['var_explained'] = pca.explained_variance_ratio_.sum()
    pca_results.append(res)
df_pca = pd.DataFrame(pca_results)

# --- (d) Time lag ---
lag_results = []
for lag in [0, 1, 2, 3, 5, 10]:
    tl_copy = timeline.copy()
    if lag > 0:
        for col in ['steps', 'steps_roll_3', 'steps_roll_5', 'steps_roll_10',
                    'step_diff', 'steps_std_5', 'steps_max_5']:
            if col in tl_copy.columns:
                tl_copy[col] = tl_copy[col].shift(lag)

    lag_train = tl_copy[tl_copy['bpm'].notna()].copy()
    lag_train['bpm_last_known'] = tl_copy['bpm'].shift(1).ffill().
↪loc[tl_copy['bpm'].notna()]
    lag_train['bpm_next_known'] = tl_copy['bpm'].shift(-1).bfill().
↪loc[tl_copy['bpm'].notna()]
    ki = lag_train.index.values
    dl = np.zeros(len(ki), dtype=int)
    dn = np.zeros(len(ki), dtype=int)
    for i in range(len(ki)):
        if i > 0: dl[i] = ki[i] - ki[i-1]
        if i < len(ki)-1: dn[i] = ki[i+1] - ki[i]
    lag_train['dist_to_last_bpm'] = dl
    lag_train['dist_to_next_bpm'] = dn
    td = (lag_train['dist_to_last_bpm'] + lag_train['dist_to_next_bpm']).
↪replace(0, 1)
    lag_train['bpm_neighbor_interp'] = (
        lag_train['bpm_last_known'] * (1 - lag_train['dist_to_last_bpm']/td) +
        lag_train['bpm_next_known'] * (1 - lag_train['dist_to_next_bpm']/td)
    )
    lag_train['gap_length'] = lag_train['dist_to_last_bpm'] +␣
↪lag_train['dist_to_next_bpm']

    X_lag = lag_train[feature_cols].values
    y_lag = lag_train['bpm'].values
    v = ~np.any(np.isnan(X_lag), axis=1)
    X_lag, y_lag = X_lag[v], y_lag[v]
```

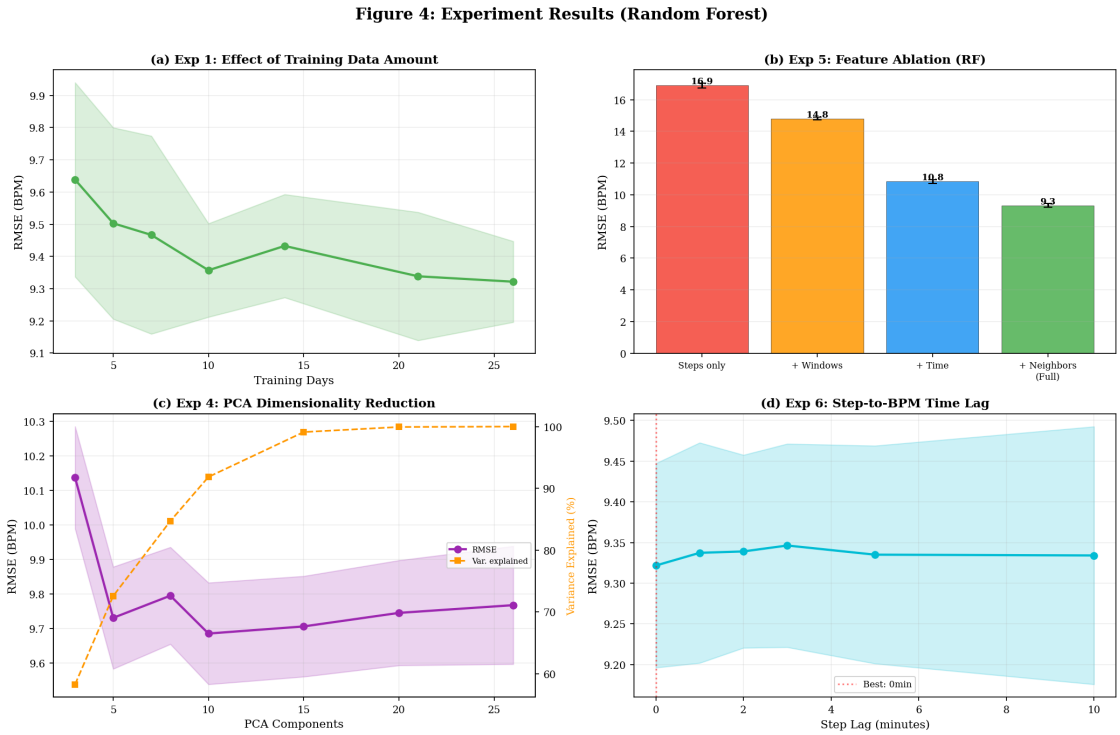
```
    if len(X_lag) > 50:
        res = evaluate_model(make_rf(), X_lag, y_lag, cv=5, name=f'{lag}min')
        res['lag'] = lag
        lag_results.append(res)

df_lag = pd.DataFrame(lag_results)
print('Experiment data ready')
print(f'  Amount: {len(df_amount)} configs')
print(f'  Ablation: {len(df_ablation)} groups')
print(f'  PCA: {len(df_pca)} configs')
print(f'  Lag: {len(df_lag)} configs')
```

```
Experiment data ready
  Amount: 7 configs
  Ablation: 4 groups
  PCA: 7 configs
  Lag: 6 configs
```

[8]:
```
fig, axes = plt.subplots(2, 2, figsize=(14, 9))

# (a) Data amount
ax = axes[0, 0]
ax.plot(df_amount['n_days'], df_amount['rmse_mean'], 'o-', color='#4CAF50',
        linewidth=2, markersize=6)
ax.fill_between(df_amount['n_days'],
                df_amount['rmse_mean'] - df_amount['rmse_std'],
                df_amount['rmse_mean'] + df_amount['rmse_std'],
                alpha=0.2, color='#4CAF50')
ax.set_xlabel('Training Days')
ax.set_ylabel('RMSE (BPM)')
ax.set_title('(a) Exp 1: Effect of Training Data Amount', fontweight='bold')
ax.grid(True, alpha=0.2)

# (b) Feature ablation
ax = axes[0, 1]
x_pos = range(len(df_ablation))
bars = ax.bar(x_pos, df_ablation['rmse_mean'], color=['#F44336', '#FF9800',␣
 ↪'#2196F3', '#4CAF50'],
              alpha=0.85, edgecolor='black', linewidth=0.3)
ax.errorbar(x_pos, df_ablation['rmse_mean'], yerr=df_ablation['rmse_std'],
            fmt='none', ecolor='black', capsize=4)
for bar, val in zip(bars, df_ablation['rmse_mean']):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.1,
            f'{val:.1f}', ha='center', fontsize=8, fontweight='bold')
ax.set_xticks(x_pos)
ax.set_xticklabels(df_ablation['name'], fontsize=8)
ax.set_ylabel('RMSE (BPM)')
```

```python
ax.set_title('(b) Exp 5: Feature Ablation (RF)', fontweight='bold')
ax.grid(True, alpha=0.2, axis='y')

# (c) PCA
ax = axes[1, 0]
ax.plot(df_pca['n_components'], df_pca['rmse_mean'], 'o-', color='#9C27B0',
        linewidth=2, markersize=6, label='RMSE')
ax.fill_between(df_pca['n_components'],
                df_pca['rmse_mean'] - df_pca['rmse_std'],
                df_pca['rmse_mean'] + df_pca['rmse_std'],
                alpha=0.2, color='#9C27B0')
ax2_twin = ax.twinx()
ax2_twin.plot(df_pca['n_components'], df_pca['var_explained'] * 100, 's--',
              color='#FF9800', linewidth=1.5, markersize=5, label='Var.␣
 ↪explained')
ax2_twin.set_ylabel('Variance Explained (%)', color='#FF9800', fontsize=9)
ax.set_xlabel('PCA Components')
ax.set_ylabel('RMSE (BPM)')
ax.set_title('(c) Exp 4: PCA Dimensionality Reduction', fontweight='bold')
ax.grid(True, alpha=0.2)
lines1, labels1 = ax.get_legend_handles_labels()
lines2, labels2 = ax2_twin.get_legend_handles_labels()
ax.legend(lines1 + lines2, labels1 + labels2, fontsize=7, loc='center right')

# (d) Time lag
ax = axes[1, 1]
ax.plot(df_lag['lag'], df_lag['rmse_mean'], 'o-', color='#00BCD4',
        linewidth=2, markersize=6)
ax.fill_between(df_lag['lag'],
                df_lag['rmse_mean'] - df_lag['rmse_std'],
                df_lag['rmse_mean'] + df_lag['rmse_std'],
                alpha=0.2, color='#00BCD4')
if len(df_lag) > 0:
    best_lag = df_lag.loc[df_lag['rmse_mean'].idxmin()]
    ax.axvline(best_lag['lag'], color='red', linestyle=':', alpha=0.5,␣
 ↪label=f'Best: {int(best_lag["lag"])}min')
    ax.legend(fontsize=8)
ax.set_xlabel('Step Lag (minutes)')
ax.set_ylabel('RMSE (BPM)')
ax.set_title('(d) Exp 6: Step-to-BPM Time Lag', fontweight='bold')
ax.grid(True, alpha=0.2)

plt.suptitle('Figure 4: Experiment Results (Random Forest)', fontsize=14,␣
 ↪fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('Charts/report_fig4_experiments.png', dpi=200, bbox_inches='tight')
plt.show()
```

```
print('Saved: Charts/report_fig4_experiments.png')
```



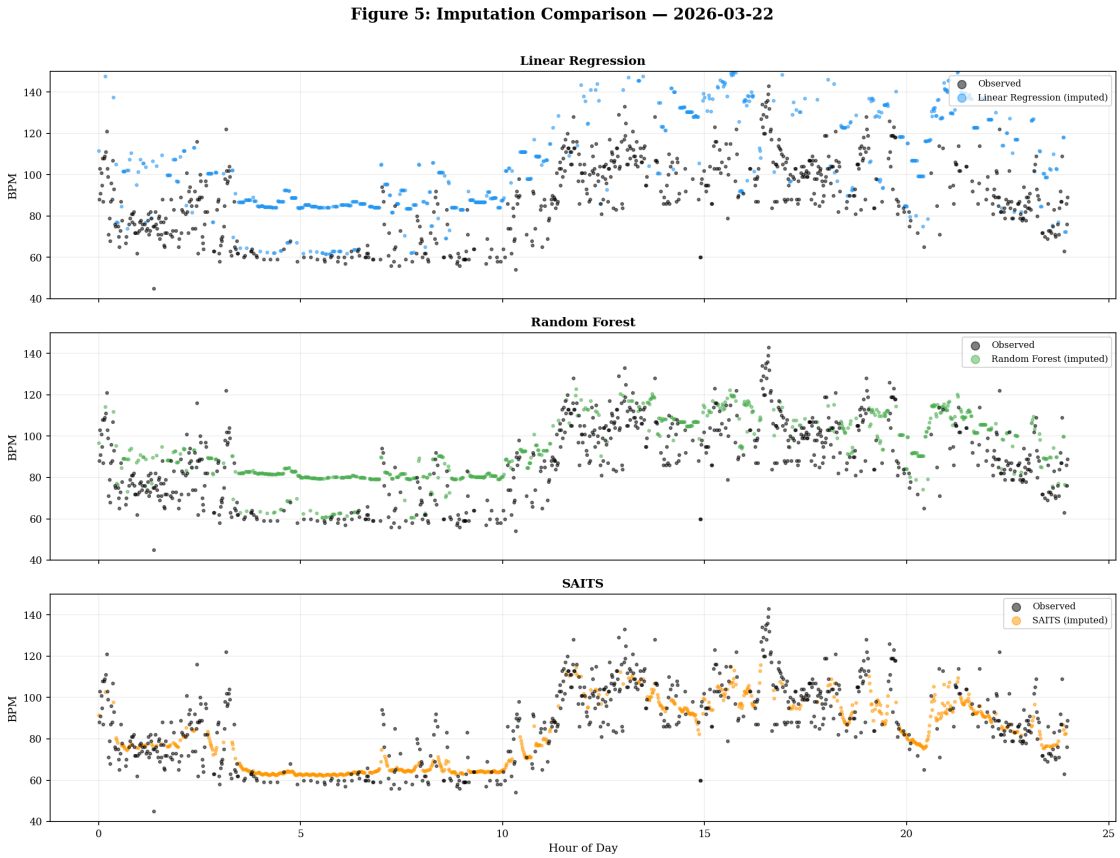

```
Saved: Charts/report_fig4_experiments.png
```

1.5 Figure 5: Imputation Comparison — Single Day

All 3 models on the same day, highlighting observed vs imputed

[9]:
```
# Find a good demo day
source = imputed_all if has_saits else timeline
days = source['day'].dt.date.unique()

best_day = None
best_score = 0
for d in days:
    dd = source[source['day'].dt.date == d]
    obs = dd['bpm'].notna().sum()
    miss = dd['bpm'].isna().sum()
    # Want a day with good mix
    score = min(obs, miss)  # balanced is best
    if score > best_score and obs > 100:
        best_score = score
        best_day = d
```

```python
if best_day is None:
    best_day = days[len(days)//2]

dd = source[source['day'].dt.date == best_day].copy()
hours = dd['minute'].dt.hour + dd['minute'].dt.minute / 60
obs = dd['bpm'].notna()
miss = dd['bpm'].isna()

impute_cols = [('bpm_lr', 'Linear Regression', '#2196F3'),
               ('bpm_rf', 'Random Forest', '#4CAF50')]
if has_saits:
    impute_cols.append(('bpm_saits', 'SAITS', '#FF9800'))

n_panels = len(impute_cols)
fig, axes = plt.subplots(n_panels, 1, figsize=(14, 3.5 * n_panels), sharex=True)
if n_panels == 1:
    axes = [axes]

for ax, (col, label, color) in zip(axes, impute_cols):
    ax.scatter(hours[obs], dd.loc[obs, 'bpm'], s=6, color='black', alpha=0.5,
               label='Observed', zorder=3)
    if col in dd.columns:
        ax.scatter(hours[miss], dd.loc[miss, col], s=6, color=color, alpha=0.5,
                   label=f'{label} (imputed)', zorder=2)
    ax.set_ylabel('BPM')
    ax.set_ylim(40, 150)
    ax.set_title(label, fontweight='bold', fontsize=11)
    ax.legend(loc='upper right', fontsize=8, markerscale=3)
    ax.grid(True, alpha=0.2)

axes[-1].set_xlabel('Hour of Day')

plt.suptitle(f'Figure 5: Imputation Comparison - {best_day}', fontsize=14,␣
 ↪fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('Charts/report_fig5_imputation_day.png', dpi=200,␣
 ↪bbox_inches='tight')
plt.show()
print(f'Saved: Charts/report_fig5_imputation_day.png (day: {best_day})')
```



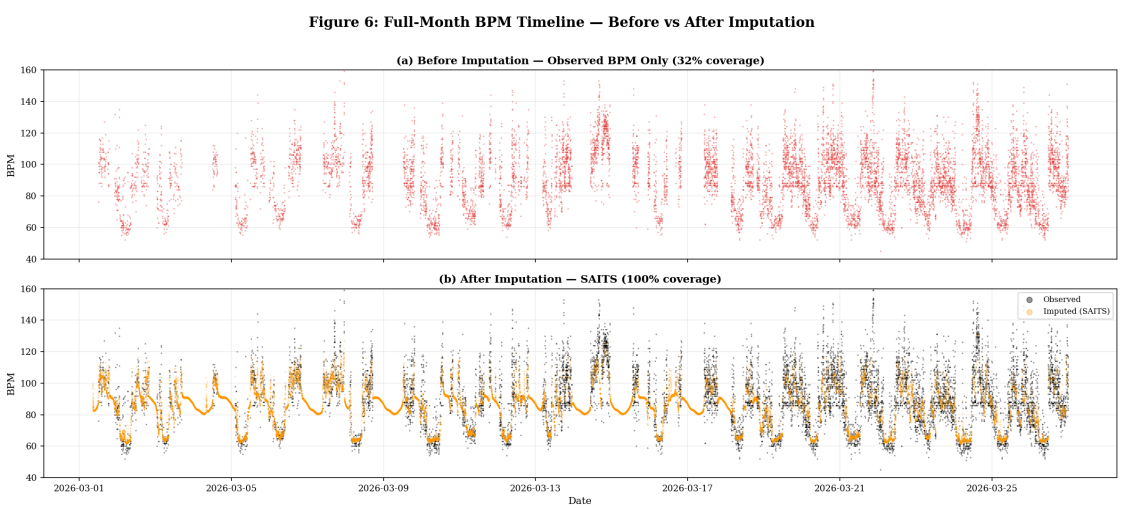
Saved: Charts/report_fig5_imputation_day.png (day: 2026-03-22)

1.6 Figure 6: Full-Month Before/After

The big picture — original sparse data vs fully imputed timeline

[10]:
```
fig, axes = plt.subplots(2, 1, figsize=(16, 7), sharex=True)

# (a) Before - observed only
ax = axes[0]
obs = timeline['bpm'].notna()
ax.scatter(timeline.loc[obs, 'minute'], timeline.loc[obs, 'bpm'],
           s=0.3, alpha=0.4, color='#E53935')
ax.set_ylabel('BPM')
ax.set_title('(a) Before Imputation - Observed BPM Only (32% coverage)',␣
 ↪fontweight='bold')
ax.set_ylim(40, 160)
ax.grid(True, alpha=0.2)

# (b) After - use best model (RF or SAITS)
ax = axes[1]
```

```
best_col = 'bpm_saits' if has_saits and 'bpm_saits' in source.columns else␣
↪'bpm_rf'
best_label = 'SAITS' if 'saits' in best_col else 'RF'

# Plot observed in black
ax.scatter(source.loc[obs, 'minute'], source.loc[obs, 'bpm'],
          s=0.3, alpha=0.4, color='black', label='Observed')
# Plot imputed in color
miss = source['bpm'].isna()
if best_col in source.columns:
    imp_color = '#FF9800' if 'saits' in best_col else '#4CAF50'
    ax.scatter(source.loc[miss, 'minute'], source.loc[miss, best_col],
              s=0.3, alpha=0.3, color=imp_color, label=f'Imputed␣
↪({best_label})')
ax.set_xlabel('Date')
ax.set_ylabel('BPM')
ax.set_title(f'(b) After Imputation - {best_label} (100% coverage)',␣
↪fontweight='bold')
ax.set_ylim(40, 160)
ax.grid(True, alpha=0.2)
ax.legend(loc='upper right', fontsize=8, markerscale=10)

plt.suptitle('Figure 6: Full-Month BPM Timeline - Before vs After Imputation',
            fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('Charts/report_fig6_before_after.png', dpi=200, bbox_inches='tight')
plt.show()
print('Saved: Charts/report_fig6_before_after.png')
```




```
Saved: Charts/report_fig6_before_after.png
```

1.7 Summary: Report Figure Inventory

Figure	File	Content	Report Section
Fig 1	`report_fig1_dataset.png`	Timeline + distribution + daily coverage	Dataset
Fig 2	`report_fig2_features.png`	Correlation + autocorrelation + circadian	Dataset / Features
Fig 3	`report_fig3_models.png`	RMSE/R^2 bars + feature importance	Results
Fig 4	`report_fig4_experiments.png`	Parameters, ablation, PCA, time lag	Experiments
Fig 5	`report_fig5_impact_by_day.png`	Simulation 3 model comparison	Results
Fig 6	`report_fig6_before_after.png`	Full-month before/after	Results

1.7.1 Notes for Report Writing (Phase 6)

- Each figure has (a)(b)(c) sub-panels — reference them specifically in text
- The individual experiment charts from Phase 3 (`exp1_*.png` etc.) can go in an appendix if needed
- Phase 4 charts (`phase4_*.png`) have more detail — use if you have space
- All charts are 200 DPI — suitable for print